

CSCE 701 CYBERSECURITY · MODULE 10 · LECTURE NOTES

Incident Response

What you do, decided before you need it.

MODULE OUTCOMES

- Explain the phases and what each requires in advance.
- Explain the containment trade and decide it.
- Explain evidence handling and why it constrains action.
- Run a tabletop exercise.
- Write a blameless post-mortem that produces change.

1 The phases

Phase	The work	What must be prepared in advance
Prepare	Everything in the rows below, before you need it.	The only phase whose timing you control — see the note.
Detect	Notice that it is happening.	Module 09's logging and detection, and a reporting channel that a person can use — many incidents are first reported by a customer or an employee.
Contain	Stop it spreading.	The authority to act and the technical means to do so (§2).
Eradicate	Remove the attacker's access.	A credential rotation procedure that has actually been executed before (Module 07 §3).
Recover	Restore service, and verify that what you restored is clean.	Tested restores and known-good images — an untested backup is a hope, and restoring a compromised image reinstates the attacker.
Learn	Change the system so it does not recur.	A blameless process that produces tracked actions (§4).

WHAT MUST EXIST BEFORE THE INCIDENT

A named person who decides, and an escalation path that functions at three in the morning on a Sunday — **including an explicit answer to who may authorise taking production down**, because that decision will be needed and nobody wants to make it unilaterally.

Out-of-band communication. If the incident is in your email or your chat system, you cannot coordinate the response inside it — the attacker is reading it — and **deciding this during the incident is far too late**. A pre-arranged alternative channel, with the numbers written down somewhere not on the compromised system.

Access to act: can the person responding actually isolate a host, revoke a credential, and retrieve logs — or do they have to wake somebody who holds the permission? **Module 03's least privilege and the need to respond are in genuine tension, and the resolution is break-glass access that is logged and alerted rather than absent.**

And the contact list: legal counsel, communications, the insurer, your regulator if you have one, and law enforcement. **Assembled calmly beforehand, rather than searched for during** — and the legal contact matters most, for §3's reasons.

2 Containment

CONTAINMENT TRADES EVIDENCE AND INTELLIGENCE AGAINST FURTHER DAMAGE

Isolate immediately and you stop the damage, lose visibility into what the attacker was doing, and inform them that they have been detected — after which they may change tactics, or act destructively.

Watch and wait and you learn the scope, the method, and the other footholds you had not found — while the damage continues, and while your knowledge of the damage grows.

And the right answer depends entirely on what is at stake: active data exfiltration or any destructive activity means contain now, without deliberation; **a single reconnaissance foothold with no evidence of movement may be worth observing briefly**, if you have the capability to observe it.

But decide it deliberately, and record the decision and the reasoning — and the default under uncertainty should be to contain, because **the downside of waiting is unbounded and the downside of containing is inconvenience**. **Organisations that chose to observe and regretted it outnumber those that contained and regretted it.**

ISOLATE network-isolate the host. DO NOT POWER IT OFF -- powering off destroys memory evidence and may trigger destructive logic on shutdown.

REVOKE sessions, tokens, and keys for every affected principal. This is where Module 07 section 3's tested rotation procedure pays for itself, and where an untested one fails.

BLOCK the attacker's known infrastructure at the egress boundary (Module 04 section 2) -- which is also why egress filtering was worth having before the incident.

DISABLE the affected feature or integration, if doing so closes the path. Removing capability is faster than fixing it.

AND PRESERVE AS YOU GO: snapshot before you change anything, and record every action with a timestamp. The timeline is the deliverable.

3 Evidence

- **Snapshot before changing anything. Disk and memory images, taken before remediation begins** — because remediation destroys the record of what happened, and the pressure to fix it immediately is exactly what causes the evidence to be lost.
- **Respect the order of volatility:** memory first, then network state and connections, then running processes, then disk. **The most volatile evidence first**, because it is gone in minutes and the disk will still be there in an hour.
- **Keep a contemporaneous timeline** of every action taken, by whom, at what time, and why — **which is also the post-mortem's raw material** (§4) and is impossible to reconstruct afterwards.
- **Preserve logs beyond the retention window explicitly**, before they rotate out from under the investigation — which happens during long investigations and is irrecoverable.
- **And get legal advice early** if the incident may involve personal data, a regulator, a contract, or litigation — **because the notification clocks start running without waiting for you. Several regimes require reporting within 72 hours of becoming aware**, which is a **decision to have made in advance** rather than researched on the second day.

4 Learning

A BLAMELESS POST-MORTEM ASKS WHAT MADE THE MISTAKE POSSIBLE

'Why did the engineer click the link' produces a defensive answer, an apology, and no change. 'Why did clicking a link lead to a domain compromise' produces a list of missing controls — no second factor, excessive standing privilege, flat network, no egress filtering. **The second framing is actionable and the first is not.**

So the rule is to treat human error as a *symptom* of system design — people will make mistakes at a predictable rate, so the operative question is why that mistake had that consequence (Module 11's subject).

And blamelessness is instrumental rather than kind. A blaming process produces incidents that are not reported — people conceal mistakes, near-misses go unexamined, and you lose the information you most need. **Which is the actual reason the practice exists**, and it is worth stating because 'blameless' sounds like a cultural nicety and is an information-gathering requirement.

The deliverable is a short list of specific changes with named owners and dates — and a post-mortem whose action items are not tracked to completion was an exercise in writing, which is the commonest way the learning phase fails.

- **Take a scenario directly from your threat model** (Module 01 §2) **and walk through it with the people who would actually respond** — not with a committee.
- **Ask who decides, who is called, and what they are able to do** — and **stop when somebody cannot answer**, because that is the **finding** rather than an interruption.

- **Expect to discover a missing authority, a missing access, or a missing contact** — every exercise finds at least one, and the first one typically finds several.
- **It costs two hours and carries no production risk whatsoever**, which makes it **the highest-value security exercise available to most organisations**.
- **And run it before you need it. The alternative is discovering the same gaps during a real incident**, which is the identical discovery at a vastly worse price. **Two hours, no risk, and it always finds something** — there is no cheaper security activity in this course.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
NIST SP 800-61 — Computer Security Incident Handling Guide (free)	§1's phases, in detail , with the preparation checklists worth borrowing.
Allspaw — Blameless PostMortems and a Just Culture (free)	§4's practice , short, and it changed how the industry handles incidents.
Dekker — The Field Guide to Understanding Human Error	§4's framing, from safety engineering — where the human-error-as-symptom argument originates. Library copy.
Google SRE Book — the incident management chapters (free)	§1's roles and command structure , free in full — and the incident-commander model is worth adopting.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write your preparation list** and identify which items do not yet exist.
- 2 **Determine who may authorise taking production down**, and whether that person knows.
- 3 **Arrange an out-of-band channel** and record the numbers somewhere off the primary systems.
- 4 **Check whether a responder can isolate a host and revoke a credential** without waking anyone.
- 5 **Write the containment decision criteria** for your own system, in advance.
- 6 **Practise taking a memory image** of a machine you own.
- 7 **Find out your notification obligations** and the deadline.
- 8 **Run a two-hour tabletop** on a threat from your model, and record every question nobody could answer.
- 9 **Write a blameless post-mortem** of the exercise.

10 Track the resulting actions to completion, and report how many closed.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the six phases and what each needs prepared.
- 2 Why is preparation the only phase whose timing you choose?
- 3 Give four things that must exist before an incident.
- 4 Why does out-of-band communication have to be arranged in advance?
- 5 State the containment trade and the default under uncertainty.
- 6 Why isolate rather than power off?
- 7 Give the order of volatility and why snapshots come first.
- 8 What is the surprise about notification deadlines?
- 9 Contrast the two post-mortem framings, and say why blamelessness is instrumental.
- 10 Why is a tabletop the highest-value exercise?